S T R O N G M I N D

Staff DevOps Engineer

Technical Exercise

ESTIMATED TIME	AI USAGE	SUBMISSION
~2 hours	**Encouraged**	**GitHub Repo**

Overview

Thank you for taking the time to complete this exercise. We designed it to reflect the real challenges you would encounter as a Staff DevOps Engineer at StrongMind — not to test memorized syntax or trivia, but to see how you reason about complex, production-grade problems.

You will work through a realistic scenario involving the StrongMind platform: an Azure-to-AWS service migration, CI/CD pipeline design for a Ruby on Rails application, containerization, and observability planning. The scenario is fictional but grounded in our actual tech stack.

What We Are Looking For

We are evaluating four things:

- **Thinking:** Reasoning quality
  - How you break down ambiguous problems
  - What tradeoffs you identify and how you resolve them
  - Whether your decisions account for failure modes, rollback, and operational risk
- **Depth:** Technical accuracy
  - Correct use of AWS services (ECS, ECR, RDS, CloudWatch, Secrets Manager, etc.)
  - Production-ready patterns for Docker, GitHub Actions, and IAM
- **Clarity:** Communication
  - Can you write clearly for both engineering peers and non-technical stakeholders?
  - Your ADR and observability plan should be usable as real documentation
- **Tooling:** AI fluency
  - Using AI to accelerate and improve your work is actively encouraged
  - Show good judgment in what you verify, modify, and own vs. what you trust wholesale

Ground Rules

- AI tools (ChatGPT, Claude, Copilot, etc.) are fully permitted and encouraged.
- Open internet research is allowed.

- The exercise is intentionally scoped to respect your time. Do not over-engineer — strong, clear thinking beats exhaustive volume.
- All decisions should be explainable. If a reviewer asks "why did you choose this?", your reasoning should be visible in your written deliverables.

> ℹ No AWS account is required. All deliverables are documents and configuration files — nothing needs to be deployed.
>
> Do not spend time on a working local environment. Focus your effort on design quality, reasoning, and correctness.

The Scenario

You are joining StrongMind as the Staff DevOps Engineer. Your first sprint is already queued. Two workstreams are running in parallel, and you own both.

Scenario Context: StrongMind's Current State

StrongMind runs most production workloads on AWS ECS Fargate today. The platform was built incrementally and has accumulated some debt:

- CI/CD pipelines exist but are inconsistent across repositories — some repos have robust GitHub Actions workflows, others push directly or have minimal automation.
- An on-call engineer recently said: "I can't tell if something is wrong until a customer calls." Observability is reactive, not proactive.
- Several services remain running on Azure from a legacy architecture. The most critical is the Identity Server — a .NET-based authentication service running on Azure App Service, backed by Azure SQL Database and Azure Key Vault for secrets.

Workstream A — Identity Server Migration (Azure → AWS)

The Identity Server handles authentication and token issuance for all StrongMind products. It is currently:

- Runtime: .NET 6, deployed as an Azure App Service (Linux)
- Database: Azure SQL Database (SQL Server 2019 compatibility)
- Secrets: Azure Key Vault (connection strings, signing certificates, API keys)
- Traffic: ~400 req/min sustained, peaking to ~1,200 req/min during morning school start (7–9 AM Mountain Time)
- Dependencies: Calls out to Azure AD Domain Services for directory lookups; downstream callers include the Rails LMS and PowerSchool integration

Your goal is to plan the migration of this service to AWS — containerizing it, deploying to ECS Fargate, migrating the database to RDS (SQL Server), migrating secrets to AWS Secrets Manager, and managing a zero-downtime cutover.

Workstream B — Rails CI/CD Standardization

StrongMind's primary new development is a Ruby on Rails 8 application (Ruby 3.3, PostgreSQL via RDS). Several engineering teams contribute to it. The current deployment state is inconsistent:

- Some teams have partial GitHub Actions pipelines (build only, no deploy automation)
- Deployments to ECS are sometimes triggered manually via the AWS console
- There is no automated rollback — a bad deploy requires manual intervention

Your goal is to design and write a standardized, production-grade GitHub Actions pipeline that any Rails service in the org can adopt, with automated ECS deployment and rollback on failure.

> ℹ The scenario is intentionally realistic and slightly underspecified — just like a real first week. Make reasonable assumptions where needed and document them. We are more interested in how you handle ambiguity than whether you picked the "right" answer.

Deliverables

Create a public GitHub repository and commit the following files. Structure and naming matter — treat this as if you are onboarding the team to use your work.

File	Purpose
`README.md`	Overview of your repo, assumptions made, and how to navigate the deliverables
`ADR.md`	Architecture decision record for the Identity Server migration (Part 1)
`.github/workflows/rails-deploy.yml`	GitHub Actions CI/CD pipeline for the Rails application (Part 2)
`Dockerfile`	Multi-stage production Dockerfile for the Rails application (Part 3)
`OBSERVABILITY.md`	Observability design plan for the migrated services (Part 4)

Part 1
Identity Server Migration — ADR
35 pts

Write an Architecture Decision Record (ADR) documenting your plan to migrate the Identity Server from Azure to AWS. Your ADR should be written as if it will be committed to the engineering team's knowledge base and referenced during the actual migration.

Required Sections

- Context — Describe the current state and what is driving the migration decision.
- Decision — State the migration approach you have chosen and why.

- Migration Architecture — Describe the target AWS architecture. Cover: ECS Fargate task definition design, RDS for SQL Server configuration, Secrets Manager setup (migrating from Key Vault), VPC/networking placement, IAM roles and least-privilege access.
- Traffic Cutover Strategy — Describe how you will route traffic from the Azure service to the new ECS service with zero downtime. Be specific: what does DNS cutover look like, how do you use weighted routing or blue/green, what is your rollback trigger and rollback procedure?
- Database Migration Plan — Describe how you will migrate from Azure SQL to RDS SQL Server. Address: migration tooling, any data validation approach, how you handle the cutover with minimal downtime given the 400 req/min baseline.
- Risk Register — Identify at least 3 risks, their likelihood and impact, and your mitigation for each.
- Definition of Done — What observable conditions tell you the migration is complete and successful?

> ℹ Strong ADRs are opinionated and specific. "We will use ECS Fargate" is weak. "We will deploy a Fargate task with 1 vCPU / 2 GB memory, horizontally scaled via Application Auto Scaling targeting 60% CPU, behind an ALB with /health checks every 15s with a threshold of 2" is strong.

Part 2

GitHub Actions Pipeline — Rails to ECS

35 pts

Write a production-grade GitHub Actions workflow file at .github/workflows/rails-deploy.yml that standardizes how Rails services at StrongMind are built, tested, and deployed to ECS Fargate.

Requirements

- Triggers: push to main (deploy to production), push to any branch (CI only — test and build, no deploy), and manual workflow_dispatch with an optional environment input.
- CI Jobs: Install Ruby dependencies (bundle install with caching), run RSpec tests, run Brakeman security scan, build the Docker image (using the Dockerfile from Part 3).
- Build & Push: Build multi-platform Docker image, push to ECR with both a git SHA tag and a latest tag. Use OIDC authentication to AWS (not long-lived access keys).
- Deploy: Update the ECS task definition with the new image URI, register the new task definition revision, update the ECS service (rolling or blue/green), and wait for service stability.
- Automated Rollback: If the service fails to stabilize within a timeout, automatically roll back to the previous task definition revision and send a failure notification.
- Secrets: Demonstrate correct handling of secrets — nothing sensitive hardcoded; use GitHub secrets for configuration and OIDC for AWS access.
- Concurrency: Prevent multiple deploys from running simultaneously on the same environment.

Assumptions You May Make

- ECR repository already exists. Its URI is available as a GitHub secret: ECR_REPOSITORY.
- ECS cluster name: strongmind-production. Service name: rails-app. Task definition family: rails-app.
- AWS region: us-east-1. AWS account ID is available as: AWS_ACCOUNT_ID.

- The Rails app runs database migrations as part of startup (via entrypoint) — you do not need to run them as a separate job.

Part 3

Multi-stage Dockerfile — Rails Application

15 pts

Write a production-ready multi-stage Dockerfile for a Ruby on Rails 8 application (Ruby 3.3, PostgreSQL adapter). The image should be optimized for production use on ECS Fargate.

Requirements

- Multi-stage build: use a builder stage and a final runtime stage. The runtime image should not contain build tooling, source gem caches, or unnecessary packages.
- Asset compilation: include Rails asset precompilation (assets:precompile) in the build stage.
- Security: run as a non-root user in the final stage. Do not expose unnecessary ports or files.
- Health check: include a HEALTHCHECK instruction appropriate for a Rails web process.
- Environment: configure the image to run in production mode by default, with the ability to override via environment variables passed at runtime.
- Image size: aim for a lean final image. Comment where you are making a deliberate tradeoff.

> ℹ You may choose between ruby:3.3-slim, ruby:3.3-alpine, or another base image. Document your choice with a brief comment in the Dockerfile explaining the tradeoff.

Part 4

Observability Design Plan

15 pts

Write an OBSERVABILITY.md document covering how you would instrument and monitor the Identity Server after it is migrated to AWS ECS, and the Rails application. This will serve as a working reference for the engineering team.

Required Content

- SLOs and SLIs: Define at least 2 Service Level Objectives for the Identity Server (e.g., availability, latency) with specific numeric targets and the corresponding SLIs you would measure. Explain what happens if the SLO is breached.
- Metrics and Alarms: List the key CloudWatch metrics you would monitor (with example alarm thresholds) for: ECS task health, RDS performance, and application-level signals. What would each alarm trigger?
- Distributed Tracing: Describe how you would enable AWS X-Ray for the .NET Identity Server on ECS. What would you look for in traces to diagnose a latency spike?
- Log Strategy: Describe your centralized logging approach — log groups, retention policies, and at least one useful CloudWatch Insights query that would help debug a production issue.
- Alerting Pipeline: Describe how alerts flow from CloudWatch to Jira Operations (fka OpsGenie) to the on-call engineer. Include severity levels and when you would page vs. notify.

ℹ This section is intentionally lower point-weight because we know you have less time for it. A concise, well-structured document beats an exhaustive one. Bullet lists and tables are appropriate here.

Submission

How to Submit

- Create a public GitHub repository.
- Commit all required files (see the table in the Deliverables section).
- Email the repository URL to your recruiting contact when ready.
- Your README.md should include: a brief explanation of your overall approach, any significant assumptions you made, and any areas you intentionally scoped out with a note on what you would do with more time.

A Note on AI Usage

We actively encourage you to use AI tools. The modern DevOps engineer's job is to leverage AI effectively — that means knowing when to trust generated output, when to verify it, and when to override it. We are not evaluating whether you wrote every line from memory; we are evaluating the quality of your judgment.

If you use AI to help draft sections, that is great — just make sure you can explain and stand behind every decision in the document. Reviewers may ask follow-up questions about your choices in a subsequent conversation.

Scoring Summary

Part	Points
Part 1: Identity Server Migration ADR	35
Part 2: GitHub Actions Pipeline	35
Part 3: Multi-stage Dockerfile	15
Part 4: Observability Design	15
Total	**100**

We look forward to reviewing your work. If you have any questions about the exercise, please reach out to your recruiting contact.